

Homework 2

CS 09410 and CS 09510

Topics: Chapter 2

March 1, 11.59 pm

Spring 2026

Prof. André Gomes

Read the following instructions before submitting your work:

- Submitting work copied (in part or in whole) from other persons, websites, or generative AI tools such as ChatGPT is strictly forbidden.
- Submit your answers on Canvas as a single PDF file.
- Include a heading with your name and assignment number/date.
- Handwriting is allowed if writing is readable by others.

Problem 1: Summarize the main characteristics of TCP and UDP that should be taken into account when deciding which protocol to use for an Internet application.

TCP:

- Reliable data transfer: data will be eventually delivered in an orderly manner.
- Congestion control: limits the amount of data sent concurrently to avoid throttling the network.
- Handshake: To provide reliable data transfer and congestion control, TCP needs to set up connection before start transferring data. This can cause additional delays.

UDP:

- Unreliable data transfer: best-effort approach: no guarantees packets will arrive at the destination; even when they arrive, there is no guarantee they will arrive in the same order they were sent.
- Simple and fast: The lack of features comes with benefits: no connection setup time, very small overhead compared to TCP.

Problem 2: Consider the network illustrated in Figure 1. On the left, the Internet service provider (ISP) provides Internet connectivity to approximately 5 million users located in the West coast of the US. On the right, a data center, located in the East coast, hosts the Web server for a popular newspaper. The link capacity between ISP and the data center averages 10 Gbps. The link capacity between each user and the ISP network, 100 Mbps.

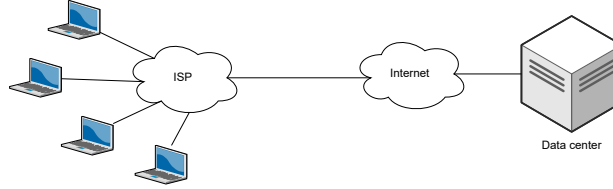


Figure 1: Example.

- a) Users have frequently complained about long delays—or even the inability—to access the newspaper website. Explain why users may be experiencing long delays.

Being users located in the West coast and server in the East coast, there are several causes of delay that may affect the users' experience, e.g., sporadic queuing delays across the network on top of non-negligible propagation delay.

- b) After some analysis, the ISP found that the demand for that newspaper website is particularly high in the mornings. Between 7 and 8 am, it handles 1,956 requests per second on average, each request being a webpage of approximately 5 Mbits. It was also found that approximately 30% of those requests are to common web pages. Someone recommended that the ISP should install a Web cache server. Would you support that recommendation? Why or why not?

Yes, that recommendation is valid and can improve users' experience. Here is why:

Without a Web cache server, the link utilization of the link between ISP and the Internet between 7-8 am averages:

$$u = \frac{1956 \times 5 \times 10^6}{10 \times 10^9} = 0.978, \quad (1)$$

which is high and likely to incur in long queueing delays as well as packet loss.

Since approximately 30% of the web page requests are the same, the ISP can reduce the traffic on that link by a factor of 0.7 by installing a Web cache server. The new link utilization is:

$$u = \frac{1956 \times 5 \times 10^6 \times 0.7}{10 \times 10^9} = 0.6846, \quad (2)$$

which is likely to improve the users' browsing experience in the morning.

Problem 3: What is the HOL blocking issue in HTTP/1.1 and how does HTTP/2 attempt to solve it?

The HOL blocking problem is a large file in the webpage (i.e. video clip) will take a long time to pass through the bottleneck link such that it blocks other small objects behind it. HTTP/2 solves this problem by breaking each message into small frames, and interleave the request and response messages on the same TCP connection.

Problem 4: True or false:

- a) A user requests a web page that consists of some text and three images. For this page, the client will send one request message and receive four response messages.
- b) Two distinct web pages can be sent over the same persistent connection.
- c) With non-persistent connections between browser and server, it is possible for a single TCP segment to carry two distinct HTTP request messages.
- d) The **Date:** header in HTTP response message indicates when the object in the response was last modified.
- e) HTTP response messages never have an empty message body.

F, T, F, F, F

Problem 5: Consider an HTTP client that wants to retrieve a web document at a given URL. The IP address of the HTTP server is initially unknown. What transport and application-layer protocols besides HTTP are needed in this scenario?

Application layer protocols: DNS and HTTP

Transport layer protocols: UDP for DNS; TCP for HTTP

Problem 6: Consider a short, 10-meter link connecting a client and a server. The link supports communication at the 150 bits/sec rate in both directions.

Now consider the HTTP protocol. The client requests a 100 Kbits object that contains 10 referenced objects to be downloaded from the same server. For simplicity, assume that HTTP requests are negligible compared to the ≈ 100 Kbits HTTP responses.

Let the TCP connection setup time be $600/R$ sec¹, where R is the link bandwidth available for each connection in bits/sec. How long does it take to download the entire set of objects?

Note: Ultimately, the time it takes to download the entire set of objects involves propagation and transmission delays corresponding to (1) setting up a TCP connection, (2) sending HTTP requests, and (3) receiving HTTP responses. That said, I'm ignoring the propagation delay and the time it takes to send HTTP requests based on the following:

- It's a short link without any information about routers or switches, and thus, I'm assuming its point-to-point. For a 10-meter link with signals propagating at speeds close to the speed of light (e.g., $\approx 2 \times 10^8$ m/sec), you can easily show that the propagation delay is negligible compared to the transmission delay in this setup.
- HTTP transactions are two-way, request and response with object. Since nothing has been said about the size of requests, it's fair to assume that HTTP requests are much smaller than HTTP objects. Thus, I'm assuming that downloading objects dominates (HTTP responses) and considering the delay for sending a request to be negligible.

a) Assuming non-persistent HTTP with 1 TCP connection.

$$R = 150 \text{ bits/sec.}$$

$$\text{For each object, } d_{\text{obj}} = \text{TCP setup} + \text{trans. delay} = \frac{600}{150} + \frac{100,000}{150}$$

$$\text{Total delay} = 11 \times d_{\text{obj}} \approx 7,377 \text{ sec}$$

b) Now assume that objects can be downloaded in N parallel TCP connections, and that each connection gets $1/N$ of the link bandwidth.

$$\text{First object: } d_{\text{1st}} = \frac{600}{150} + \frac{100,000}{150}.$$

For each of the subsequent 10 objects, $d_{\text{other}} = \frac{600}{15} + \frac{100,000}{15}$. Note that each connections gets 15 bits/sec. However, these are sent in parallel!

$$\text{Total delay} = d_{\text{1st}} + d_{\text{other}} \approx 7,377 \text{ sec.}$$

c) Assume persistent HTTP with 1 TCP connection.

$$\text{First object: } d_{\text{1st}} = \frac{600}{150} + \frac{100,000}{150}$$

¹For your information only: The underlying assumption is that each handshake message is 200 bits long, and it takes three messages (3-way handshake!) to setup a TCP connection.

Subsequent objects: $d_{\text{others}} = 10 \times \frac{100,000}{150}$. Note: There is no TCP setup here—same connection.

Total delay = $d_{\text{1st}} + d_{\text{others}} \approx 7,337$ sec.

Does persistent HTTP always offer significant gains compared to non-persistent HTTP?

No. As shown in the scenario above, if the TCP setup time is substantially smaller than the transmission time, persistent HTTP does not always offer significant gains compared to non-persistent HTTP.

Problem 7: In this problem, you will use the *dig* tool available on Unix and Linux hosts to explore the hierarchy of DNS servers. Before you start, explore the *man* page for *dig* (e.g., on the terminal, enter `man dig`). Particularly, familiarize yourself with the command: `dig +norecurse @<dns-server-ip> <name>`.

Our goal is to obtain the server IP address for `rowan.edu` by iteratively querying root DNS server, top-level domain DNS server, and authoritative DNS server.

- Start with a root DNS server. You can find the list of root DNS servers here (click to open). Pick one, e.g., 198.41.0.4. Using *dig*, obtain the list of DNS servers in the delegation chain. Keep querying them until you obtain the IP address for `rowan.edu`. Report the chain of requests (screenshots!) you made and the obtained IP address for `rowan.edu`.
- Repeat the steps above for a website of your choice.

Figure 2 shows the chain of queries, from left to right, top to bottom. The obtained IP ad

```

gomesa@Robigome2025 ~ % dig +norecurse @198.41.0.4 rowan.edu
; <<> DiG 9.10.6 <<> +norecurse @198.41.0.4 rowan.edu
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 40824
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 13, ADDITIONAL: 27

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;rowan.edu.                IN      A

;; AUTHORITY SECTION:
edu.        172800 IN      NS      d.edu-servers.net.
edu.        172800 IN      NS      b.edu-servers.net.
edu.        172800 IN      NS      f.edu-servers.net.
edu.        172800 IN      NS      h.edu-servers.net.
edu.        172800 IN      NS      l.edu-servers.net.
edu.        172800 IN      NS      j.edu-servers.net.
edu.        172800 IN      NS      e.edu-servers.net.
edu.        172800 IN      NS      c.edu-servers.net.
edu.        172800 IN      NS      a.edu-servers.net.
edu.        172800 IN      NS      g.edu-servers.net.
edu.        172800 IN      NS      i.edu-servers.net.
edu.        172800 IN      NS      m.edu-servers.net.
edu.        172800 IN      NS      k.edu-servers.net.

;; ADDITIONAL SECTION:
d.edu-servers.net. 172800 IN      A      192.31.80.30
d.edu-servers.net. 172800 IN      AAAA   2001:500:856e::30
b.edu-servers.net. 172800 IN      A      192.33.14.30
b.edu-servers.net. 172800 IN      AAAA   2001:503:231d::2:30
f.edu-servers.net. 172800 IN      A      192.35.51.30
f.edu-servers.net. 172800 IN      AAAA   2001:503:d414::30
h.edu-servers.net. 172800 IN      A      192.54.112.30
h.edu-servers.net. 172800 IN      AAAA   2001:502:8cc::30

gomesa@Robigome2025 ~ % dig +norecurse @192.31.80.30 rowan.edu
; <<> DiG 9.10.6 <<> +norecurse @192.31.80.30 rowan.edu
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 11232
;; flags: qr; QUERY: 1, ANSWER: 0, AUTHORITY: 5, ADDITIONAL: 3

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;rowan.edu.                IN      A

;; AUTHORITY SECTION:
rowan.edu.        172800 IN      NS      ns5.dnsmadeeasy.com.
rowan.edu.        172800 IN      NS      ns6.dnsmadeeasy.com.
rowan.edu.        172800 IN      NS      ns7.dnsmadeeasy.com.
rowan.edu.        172800 IN      NS      ns3.rowan.edu.
rowan.edu.        172800 IN      NS      ns4.rowan.edu.

;; ADDITIONAL SECTION:
ns3.rowan.edu.    172800 IN      A      150.250.2.11
ns4.rowan.edu.    172800 IN      A      150.250.2.12

;; Query time: 12 msec
;; SERVER: 192.31.80.30#53(192.31.80.30)
;; WHEN: Thu Sep 18 16:00:25 EDT 2025
;; MSG SIZE rcvd: 175

gomesa@Robigome2025 ~ % dig +norecurse @150.250.2.11 rowan.edu
; <<> DiG 9.10.6 <<> +norecurse @150.250.2.11 rowan.edu
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 55682
;; flags: qr aa; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;rowan.edu.                IN      A

;; ANSWER SECTION:
rowan.edu.        28800  IN      A      150.250.77.84

;; Query time: 5 msec
;; SERVER: 150.250.2.11#53(150.250.2.11)
;; WHEN: Thu Sep 18 16:01:06 EDT 2025
;; MSG SIZE rcvd: 54
gomesa@Robigome2025 ~ %

```

Figure 2: DSN lookup chain for `rowan.edu`.